

Price and Quality Comparison Application (PQC)



By

Ahmad Masood

Supervised By

Dr. Khalid Saleem

DEPARTMENT OF COMPUTER SCIENCE
QUAID-I-AZAM UNIVERSITY, ISLAMABAD

APPROVAL PAGE

This document entitled **Price and Quality Comparison Application**, submitted by **Ahmad Masood** and has been approved by the supervisor for submission to the Department of Computer Sciences, Quaid-i-Azam University Islamabad.

Program: BS Computer Sciences

Submitted By:

Name: Ahmad Masood

Reg No: 04072213041

Student's Signature: _____

Submitted To:

Supervisor's Signature: _____

Name: Dr. Khalid Saleem

Date: _____

Official Stamp: _____

ACKNOWLEDGEMENT

First and foremost, I bow my head in gratitude to Allah Almighty, the most beneficent and merciful, for bestowing upon me the strength, knowledge, and ability to complete this Final Year Project.

I would like to express my deep gratitude to my supervisor, Dr. Khalid Saleem, for his patient guidance, enthusiastic encouragement, and useful critiques of this research work. His insightful feedback and technical advice regarding the complexities of web scraping and system architecture were invaluable in keeping the project on track.

My special thanks go to the Department of Computer Science for providing the necessary resources and environment to complete this project. Finally, I thank my parents and friends for their prayers and moral support throughout this journey.

ABSTRACT

The **Price and Quality Comparison** Application (PQC) is a web-based information system designed to revolutionize how consumers in Pakistan discover and compare products across both online and physical retail chains. In the current fragmented market, comparing prices between local supermarkets (like Imtiaz or Punjab Cash & Carry) and online stores (like PriceOye or Daraz) is a manual and difficult process. This project addresses this gap by developing a specialized Web Scraping Engine using Puppeteer and Node.js that retrieves real-time pricing, availability, and quality ratings from target vendor websites. The system features a responsive frontend built with Next.js and utilizes Natural Language Processing (NLP) to handle Urdu-to-English translation, ensuring accessibility for a wider audience. This documentation details the software development lifecycle, including the architectural design, scraping logic, and database management required to build this centralized comparison platform.

Contents

1	Software Project Management Plan (SPMP)	1
1.1	INTRODUCTION	2
1.2	Project Overview	2
1.2.1	Motivation	2
1.2.2	Application	2
1.2.3	Existing Systems	2
1.3	Project Deliverables	3
1.4	Project Organization	3
1.4.1	Software Process Model	3
1.4.2	Roles and Responsibilities	4
1.4.3	Tools and Techniques	4
1.5	Project Management Plan	4
1.5.1	Tasks	5
1.5.2	Description	5
1.6	Gantt Chart	6
1.7	Summary	8
2	Software Requirements Specifications (SRS)	9
2.1	Introduction	10
2.2	Product Overview	10
2.3	Specific Requirements	10
2.3.1	Functional Requirements	10
2.3.2	Non-Functional Requirements	11
2.4	Software Product Features	12
2.5	Software System Attributes	12
2.6	List of Use Cases	12
2.7	Use Case Diagram	13
2.8	Use Case Descriptions	13
2.9	System Sequence Diagram	18
2.10	Activity Diagrams	21
2.11	Domain Model	26

3	Software Design Description (SDD)	27
3.1	Introduction	28
3.1.1	Design Overview	28
3.1.2	Design Patterns Implementation	28
3.1.3	Requirements Traceability Matrix (RTM)	28
3.2	System Architecture Design	29
3.2.1	Architectural Overview	29
3.2.2	Architectural Layers	29
3.2.3	System Components Interaction	30
3.3	System Sequence Diagram	31
3.4	Class Diagram	34
4	Software Implementation (SI)	35
4.1	System Definition	36
4.2	Development Tools	36
4.3	Implementation Details	36
4.3.1	User Interaction Module	37
4.3.2	Query Processing (NLP) Module	37
4.3.3	Web Scraping and Data Retrieval Module	37
4.3.4	Product Matching and Comparison	37
4.4	Data Flow	37
4.5	Summary	38
5	Software Test Documentation (STD)	39
5.1	Introduction	40
5.1.1	Test Approach	40
5.2	Test Plan	40
5.2.1	Testing Objectives	40
5.2.2	Testing Scope	40
5.2.3	Testing Tools and Environment	41
5.2.4	Testing Schedule	41
5.3	Test Cases	41
5.3.1	Test Case: Valid Product Search	41
5.3.2	Performance Testing	42
5.3.3	Test Case: View Product Details	42
5.3.4	Test Case: Product Comparison	42
5.3.5	Test Execution Summary	43
5.4	Conclusion	43
6	Conclusion and Future Work	44

6.1	Conclusion	45
6.2	Future Work	45

List of Tables

2.1	Use Case Description: Search Product	14
2.2	Use Case Description: View Product Details	15
2.3	Use Case Description: View Product Comparison	16
2.4	Use Case Description: Filter and Sort Products	17
3.1	Requirements Traceability Matrix	28
5.1	Testing Schedule	41
5.2	Functional Test Case: Valid Product Search	42
5.3	Test Case: Single Store Response Time	42
5.4	Functional Test Case: View Product Details	42
5.5	Functional Test Case: Product Comparison	43
5.6	Overall Test Execution Summary	43

List of Figures

1.1	Iterative Model	3
1.2	Gantt Chart	6
1.3	Gantt Chart Timeline	6
1.4	Gantt Chart	7
1.5	Gantt Chart Timeline	8
2.1	Use case Diagram	13
2.2	System Sequence Diagram: Search Product	18
2.3	System Sequence Diagram: View Product Details	19
2.4	System Sequence Diagram: View Product Comparison	20
2.5	System Sequence Diagram: Filter And Sort	21
2.6	Activity Diagram: Search Product	22
2.7	Activity Diagram: View Product Details	23
2.8	Activity Diagram: View Product Comparison	24
2.9	Activity Diagram: Filter And Sort	25
2.10	Domain Diagram	26
3.1	Three- Tier Architecture	29
3.2	Sequence Diagram: Search Product	31
3.3	Sequence Diagram: View Product Details	32
3.4	Sequence Diagram: View Product Comparison	32
3.5	Sequence Diagram: Filter And Sort	33
3.6	Class Diagram	34

CHAPTER 1 Software Project Manage- ment Plan (SPMP)

1.1 INTRODUCTION

1.2 Project Overview

Price and Quality Comparison Application (PQC) is a web-based system which is meant to be gathers product information on various sources in a single platform. Instead of compelling the users to browse through various websites.

Firstly, a user inputs a name of a product either in English or Urdu. The system interprets this input through Natural Language Processing. A web scraping engine gathers product information on different online sources and accessible digital brick-and-mortar stores. This data is organized. The arrangement of prices, ratings, and availability among others is designed in a manner it makes comparison presentable and not perplexing. The users are able to scan quickly choose, investigate, and find out what is most appropriate to them.

1.2.1 Motivation

In the modern online market, it has become disturbingly hard to find the appropriate product. Users tend to use various sites to check out various prices and are still left in the doldrums about quality.

The concept behind the Price and Quality Comparison Application (PQC) arises out of this very gap. The aim is basic yet purposeful: to minimize the effort, eliminate the confusion and provide users clarity. It can be used to integrate smart search (with NLP) and automated data gathering the system is designed to make it easy to compare products and view both price and quality side by side and decide purposefully.

1.2.2 Application

The PQC system has diverse applications across various user demographics:

- **General Consumers:** To compare the daily grocery prices in different supermarkets such as Metro. and Imtiaz.
- **Tech Buyers:** To compare electronics specifications and prices of electronics in specialized vendors like Shophive and Tejar
- **Market Analysts:** Price trends can be tracked using the historical data in our database. and availability of products in the long run

1.2.3 Existing Systems

Local e-commerce sites like Daraz have their own internal search, but they do not compare their prices with competitors. Existing systems often lack:

- **Cross-Platform Comparison:** They do not compare Physical Store prices with Online Store prices.
- **Urdu Language Support:** Most platforms strictly require English input.
- **Unit Normalization:** They fail to compare “1kg” with “1000g” accurately.

1.3 Project Deliverables

The most important PQC project deliverables are as follows:

- Search, filter, and view web application developed fully on Next.js.
- Books, Electronics and Groceries Scrape Backend API to scrape data on books, electronics and groceries websites.
- MongoDB database with the information about products and user search logs.
- Detailed documentation of the project such as the system architecture, code description and the user manuals

1.4 Project Organization

1.4.1 Software Process Model

The PQC Application will be developed under an Iterative and Incremental software. process model. Considering that web scraping is a complicated process because targeted sites are often alter their design), one category to construct the scraper (e.g.Books) initially, try out, and then increase to others (Electronics, Grocery). This ensures continuous feedback and stability.

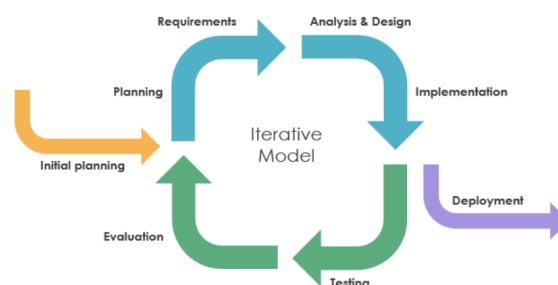


Figure 1.1: Iterative Model

1.4.2 Roles and Responsibilities

Student:

- Full stack development using Next.js for frontend and Node.js for backend APIs
- Web scraping using Puppeteer for dynamic content handling
- MongoDB database design and query optimization
- Testing and validation of scraped data accuracy
- Verification of Urdu-to-English translation functionality
- Debugging, performance optimization, and documentation

Supervisor:

- Project guidance and supervision
- Review of project progress and deliverables
- Ensuring academic and technical standards

1.4.3 Tools and Techniques

- **Development Environment:** Visual Studio Code (VS Code).
- **Frontend Framework:** Next.js (React) for a responsive UI.
- **Backend Runtime:** Node.js for handling server-side logic.
- **Automation Tool:** Puppeteer (for headless browser scraping).
- **Database:** MongoDB for flexible product data storage.
- **Version Control:** Git and GitHub.

1.5 Project Management Plan

This section outlines the planning, main tasks, and phase-wise activities of the Product Comparison System.

1.5.1 Tasks

The project is divided into the following high-level tasks:

1. Requirement Gathering and Store Selection
2. System Architecture Design
3. Development of Scraping Modules
4. Frontend Development and Integration
5. Testing and Validation
6. Final Documentation

1.5.2 Description

Requirements and Analysis Phase

This step was dedicated to determining the key product features to be compared (e.g., title, price, rating, availability) and choosing target websites.

Activities:

- Website structure analysis.
- Determinism of quality rating and unit conversion.

Design Phase

The system blueprint and workflow were developed in the design stage.

Activities:

- System architecture and ERD design.
- Development of wireframes on search and comparison pages.

Development Phase:

Applied standard core system operations and encompassed frontend elements.

Testing and Validation Phase

Assured accuracy of data, adequate comparison output and efficient functionality of the system.

1.6 Gantt Chart

	①	Name	Duration	Start	Finish	Predecessors
1		Project Planning	87 days?	11/27/25, 8:00 AM	3/27/26, 5:00 PM	
2		Process	3 days?	11/27/25, 8:00 AM	12/1/25, 5:00 PM	
3		Software Project Management Plan (SPMP)	11 days?	11/27/25, 8:00 AM	12/11/25, 5:00 PM	
4		Introduction	2 days?	11/27/25, 8:00 AM	11/28/25, 5:00 PM	
5		Project overview	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
6		Motivation	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
7		Application	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
8		Existing Systems	1 day?	11/27/25, 8:00 AM	11/27/25, 5:00 PM	
9		Project Deliverables	2 days?	11/27/25, 8:00 AM	11/28/25, 5:00 PM	
10		Project Organization	6 days?	12/1/25, 8:00 AM	12/8/25, 5:00 PM	4
11		Software Process Model	3 days?	12/1/25, 8:00 AM	12/3/25, 5:00 PM	
12		Roles and Responsibilities	1 day?	12/4/25, 8:00 AM	12/4/25, 5:00 PM	11
13		Tools and Techniques	2 days?	12/5/25, 8:00 AM	12/8/25, 5:00 PM	12
14		Project Management Plan	3 days?	12/9/25, 8:00 AM	12/11/25, 5:00 PM	10
15		Tasks	1 day?	12/9/25, 8:00 AM	12/9/25, 5:00 PM	
16		Description	2 days?	12/10/25, 8:00 AM	12/11/25, 5:00 PM	15
17		Summary	1 day?	12/9/25, 8:00 AM	12/9/25, 5:00 PM	
18		M1:MileStone Completes	0 days?	12/11/25, 5:00 PM	12/11/25, 5:00 PM	16
19		Software Requirements Specifications (SRS)	37 days?	12/12/25, 8:00 AM	2/2/26, 5:00 PM	18
20		Introduction	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
21		Overview	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
22		Requirements	4 days?	12/12/25, 8:00 AM	12/17/25, 5:00 PM	
23		Functional Requirements	2 days?	12/12/25, 8:00 AM	12/15/25, 5:00 PM	
24		Non-Functional Requirements	2 days?	12/16/25, 8:00 AM	12/17/25, 5:00 PM	23
25		Software Product Features	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
26		Software System Attributes	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
27		List of Use Cases	1 day?	12/12/25, 8:00 AM	12/12/25, 5:00 PM	
28		Use Case Diagram	7 days?	12/12/25, 8:00 AM	12/22/25, 5:00 PM	
29		Use Case description	9 days?	12/23/25, 8:00 AM	1/2/26, 5:00 PM	28
30		System sequence Diagram	13 days?	1/5/26, 8:00 AM	1/21/26, 5:00 PM	29
31		Activity Diagram	8 days?	1/22/26, 8:00 AM	2/2/26, 5:00 PM	30
32		M2:MileStone Completes	0 days?	2/2/26, 5:00 PM	2/2/26, 5:00 PM	31
33		Software Design Description	11 days?	2/3/26, 8:00 AM	2/17/26, 5:00 PM	32

FYP-1 - page1

Figure 1.2: Gantt Chart

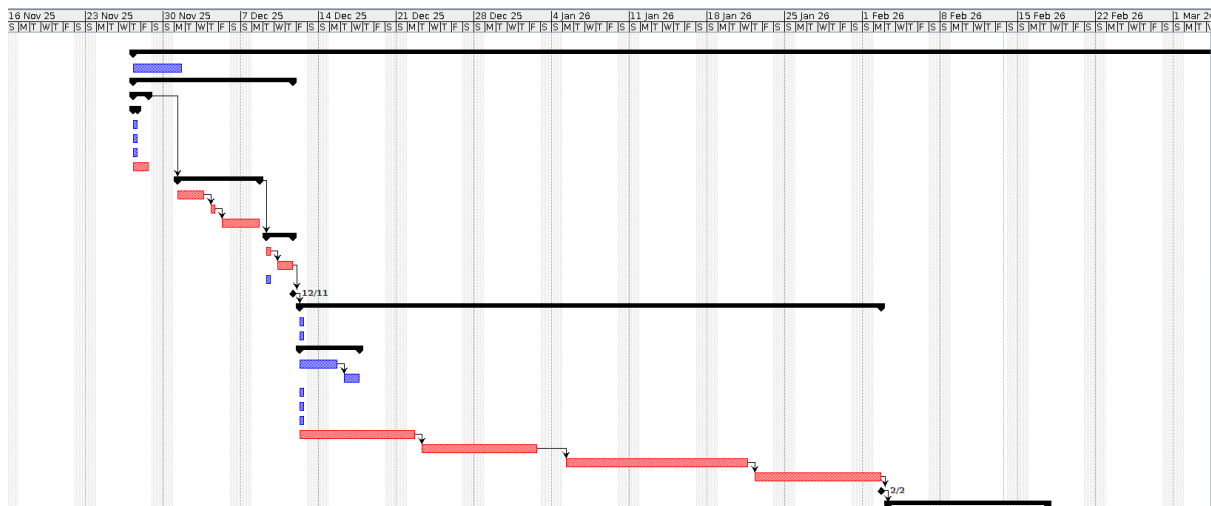


Figure 1.3: Gantt Chart Timeline

33	Software Design Description	11 days?	2/3/26, 8:00 AM	2/17/26, 5:00 PM	32
34	Introduction	2 days?	2/3/26, 8:00 AM	2/4/26, 5:00 PM	
35	Design Overview	1 day?	2/3/26, 8:00 AM	2/3/26, 5:00 PM	
36	Design Patterns Implementation	1 day?	2/3/26, 8:00 AM	2/3/26, 5:00 PM	
37	Requirements Traceability Matrix	2 days?	2/3/26, 8:00 AM	2/4/26, 5:00 PM	
38	System Architecture Design	2 days?	2/5/26, 8:00 AM	2/6/26, 5:00 PM	34
39	Architectural Overview	1 day?	2/5/26, 8:00 AM	2/5/26, 5:00 PM	
40	Architectural Layers	2 days?	2/5/26, 8:00 AM	2/6/26, 5:00 PM	
41	System Components Interaction	1 day?	2/5/26, 8:00 AM	2/5/26, 5:00 PM	
42	Class Diagram	4 days?	2/9/26, 8:00 AM	2/12/26, 5:00 PM	38
43	Domain Modal	3 days?	2/13/26, 8:00 AM	2/17/26, 5:00 PM	42
44	M3:MileStone Completes	0 days?	2/17/26, 5:00 PM	2/17/26, 5:00 PM	43
45	Software Implementation (SI)	10 days?	2/18/26, 8:00 AM	3/3/26, 5:00 PM	44
46	System Definition	1 day?	2/18/26, 8:00 AM	2/18/26, 5:00 PM	
47	Development Tools	1 day?	2/18/26, 8:00 AM	2/18/26, 5:00 PM	
48	Implementation Details	8 days?	2/18/26, 8:00 AM	2/27/26, 5:00 PM	
49	User Interaction Module	2 days	2/18/26, 8:00 AM	2/19/26, 5:00 PM	
50	Query Processing (NLP) Module	2 days	2/20/26, 8:00 AM	2/23/26, 5:00 PM	49
51	Web Scraping and Data Retrieval Module	2 days	2/24/26, 8:00 AM	2/25/26, 5:00 PM	50
52	Product Matching and Comparison Engine	2 days	2/26/26, 8:00 AM	2/27/26, 5:00 PM	51
53	Data Flow	1 day?	3/2/26, 8:00 AM	3/2/26, 5:00 PM	48
54	Summary	1 day?	3/3/26, 8:00 AM	3/3/26, 5:00 PM	53
55	M4:MileStone Completes	0 days?	3/3/26, 5:00 PM	3/3/26, 5:00 PM	54
56	Software Test Documentation (STD)	12 days?	3/4/26, 8:00 AM	3/20/26, 8:00 AM	55
57	Introduction	1 day?	3/4/26, 8:00 AM	3/4/26, 5:00 PM	
58	Test Approach	1 day?	3/4/26, 8:00 AM	3/4/26, 5:00 PM	
59	Test Plan	8 days?	3/5/26, 8:00 AM	3/16/26, 5:00 PM	57
60	Testing Objectives	2 days	3/5/26, 8:00 AM	3/6/26, 5:00 PM	
61	Testing Scope	2 days	3/9/26, 8:00 AM	3/10/26, 5:00 PM	60
62	Testing Tools and Environment	2 days	3/11/26, 8:00 AM	3/12/26, 5:00 PM	61
63	Testing Schedule	2 days	3/13/26, 8:00 AM	3/16/26, 5:00 PM	62
64	Test Cases	2 days	3/17/26, 8:00 AM	3/18/26, 5:00 PM	59
65	Conclusion	1 day?	3/19/26, 8:00 AM	3/19/26, 5:00 PM	64
66	M5:MileStone Completes	0 days?	3/20/26, 8:00 AM	3/20/26, 8:00 AM	65
67	Conclusion and Future Work	4 days?	3/20/26, 8:00 AM	3/26/26, 8:00 AM	66
68	Conclusion	2 days	3/20/26, 8:00 AM	3/23/26, 5:00 PM	
69	Future Work	2 days	3/24/26, 8:00 AM	3/25/26, 5:00 PM	68
70	M6:MileStone Completes	0 days?	3/26/26, 8:00 AM	3/26/26, 8:00 AM	69

Figure 1.4: Gantt Chart

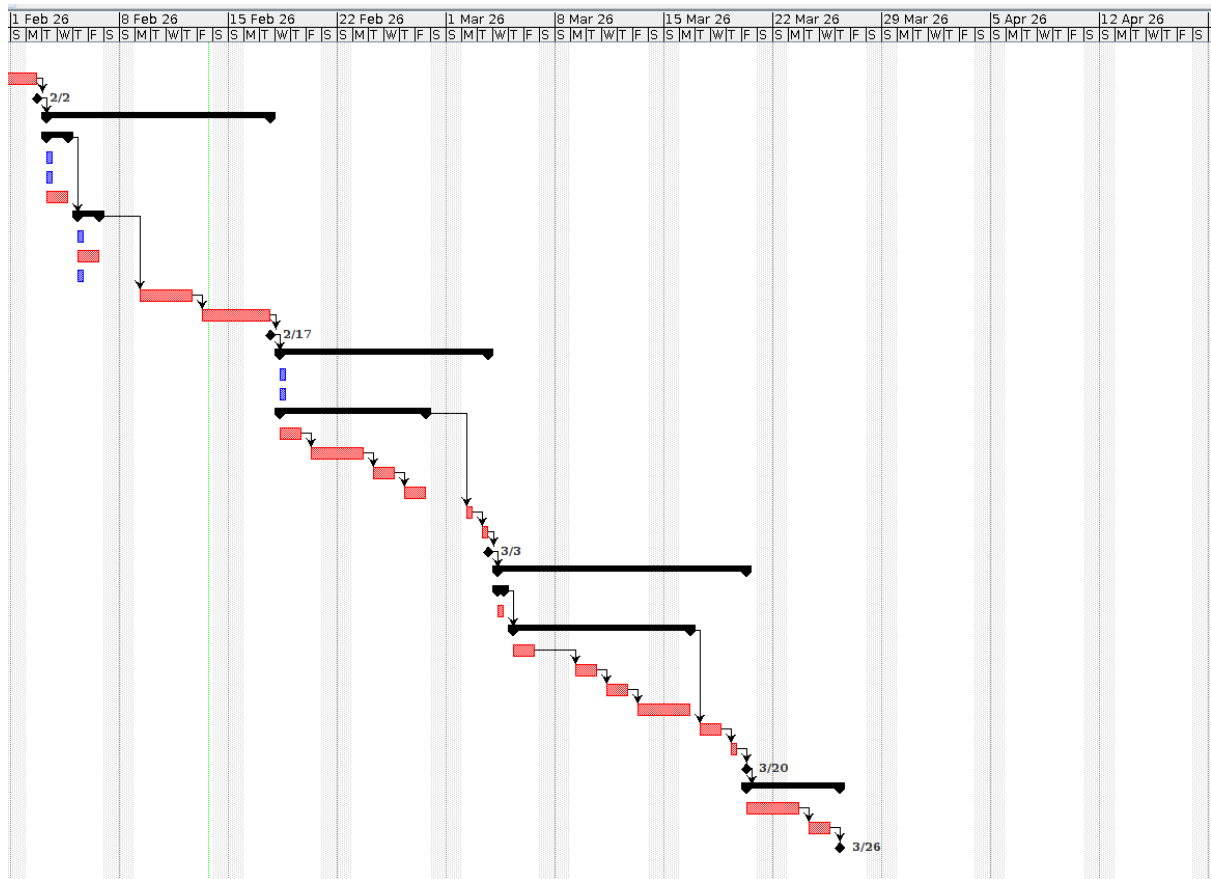


Figure 1.5: Gantt Chart Timeline

1.7 Summary

This chapter presented an overview of the Product Comparison System, including its objectives, motivation, and application scope. The development methodology, project tasks, and management plan were discussed in detail.

CHAPTER 2 Software Requirements Specifications (SRS)

2.1 Introduction

The chapter will offer a brief description of the Product Price and Quality Comparison System , with an emphasis on the system design, requirements, and functionality. It provides a structured description of the software product, its requirements, features, and system attribute . Also, the chapter gives the use cases and system sequence diagrams, which taken together show the interaction between users, system and outside parties like the Web scraping engine and NLP Module.It aimed for defining the system.

2.2 Product Overview

The Price and Quality Comparison System is a web based system that allow users to search the products, compare prices,rating among various products.It NLP component to decipher the user query and a Web Scraping Engine to obtain real-time information of various online and physical databases. The users may either search by the name or description of the item or see a detailed product. The system aggregates data to offer a convenient side by side price and quality comparison

The stakeholders are:

- **End Users (Shoppers):** People who shop are the ones using it to look up items, check differences and visit the products.
- **Retailers (Data Sources):** TThe online and physical stores, the data of whose products (prices, rating). The system crawls and compares description, and quality ratings.

2.3 Specific Requirements

The system requirements are categorized into functional and non-functional requirements:

2.3.1 Functional Requirements

1. Product Search

- The system should be an ability to input a product name or description.
- The system should be able to take queries based on an NLP module to do the translation and normalization.
- The system will have to crawl the web sources through a Web Scraping Engine in order to extract information.

2. Product Comparison

- The system needs to make a compilation of data to compare prices and quality of origin.
- Comparison data should be presented at the time when there is all-source complete data available

3. Data Filtering and Sorting

- The users should be in a position to filter the results according to category, price, or brand
- Users should be in a position to rank the results in terms of price (low-to-high) or rating.

4. Product Details

- When a user chooses a certain product, the system should show detailed information.
- The system should also be able to use the results obtained in a cache to show details effectively.

2.3.2 Non-Functional Requirements

1. Performance Requirements

- The system should be able to have real-time updates on sources.
- Filtering and sorting have to be in real-time.
- The response time taken by the system must be as low as possible in order to provide the user with a smooth experience.

2. Reliability Requirements

- The system must be able to sympathize with network failures and not crash.
- In the event that sources do not respond, they have to be considered as unavailable and not failed failing completely.

3. Usability Requirements

- The NLP module ought to deal with multilingual queries (e.g. Urdu to English).
- The results of comparisons should be understandable and easy to interpret.
- The error messages have to be easy to use (e.g. Product details unavailable).

2.4 Software Product Features

1. **Smart Search Processing:** NLP query translation and normalization.
2. **Multi-Source Scraping:** This will be an aggregation of product information across online and physical store APIs.
3. **Automated Comparison:** Automated Comparison: Comparison of the product price and quality side-by-side
4. **Caching Mechanism:** The efficient storage of search results so as to enhance the process of viewing the search results in details.

2.5 Software System Attributes

1. **Usability:** Multilingual inputs are handled in a specialized manner and the prompting of errors is clear.

2.6 List of Use Cases

The primary use cases of the system are:

1. Search Product
2. View Product Details
3. View Product Comparison
4. Filter and Sort Products

2.7 Use Case Diagram

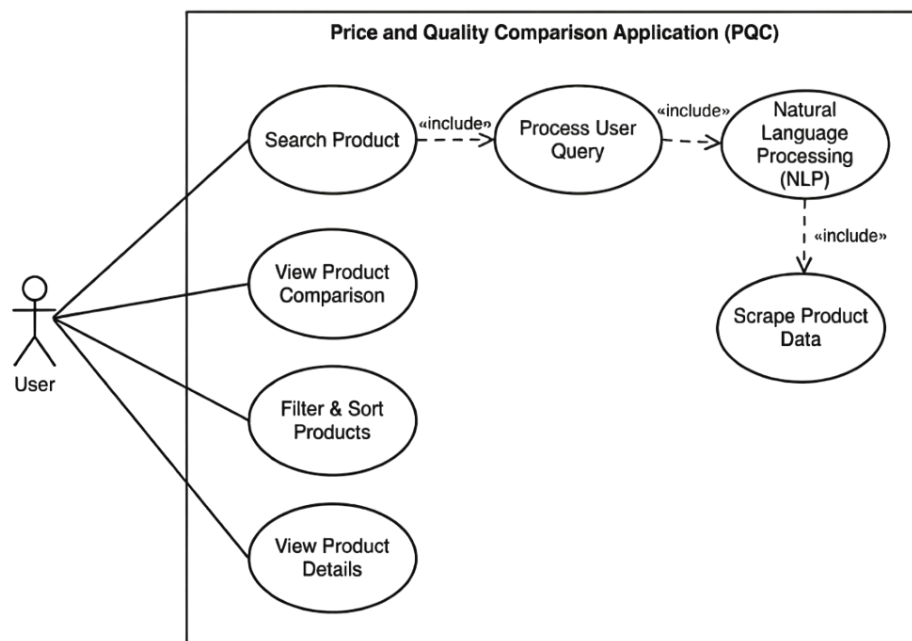


Figure 2.1: Use case Diagram

2.8 Use Case Descriptions

Use Case ID	UC-01
Use Case Name	Search Product
Primary Actor	User
Description	This allows the user to search for a product by entering its name.
Preconditions	System has access to online store APIs or scraping modules.
Postconditions	The user is presented with a list of products matching their search query, including prices, quality information.

Main Scenario	Success	1. User enters the product name or description in the search bar. 2. System sends the input to the NLP module for query processing. 3. NLP module normalizes the query. 4. System send the processed query to the Web Scraping Engine. 5. Web Scraping Engine retrieves relevant product data from various online. 6. System compiles the product data. 7. System displays the search results in a user-friendly interface.
Alternative Scenarios		• 1a. Invalid Query: User have to enter a valid product name. • 2a. NLP Failure: System prompts user to rephrase query. • 5a. No Products Found: System shows "No products found".
Frequency		High, whenever a user wants to search and compare products.

Table 2.1: Use Case Description: Search Product

Use Case ID	UC-02
Use Case Name	View Product Details
Primary Actor	User
Description	A user views detailed information of a product after selecting it from the search results.
Preconditions	• Search results have already been displayed to the user. • User has selected a product from the search results.
Postconditions	Product details are displayed to the user
Main Success Scenario	1. System displays the search results to the user. 2. User clicks on a product from the search results. 3. System checks whether the product details are available. 4. Product details are found in the DB. 5. System displays the product details to the user.
Alternative Scenarios	• 3a. Product details not available in found: System displays an error message: "Product details unavailable."
Frequency	High; whenever a user selects a product from search results.

Table 2.2: Use Case Description: View Product Details

Use Case ID	UC-03
Use Case Name	View Product Comparison
Primary Actor	User
Description	A user to view a comparison of a selected product across multiple sources.
Preconditions	• User has searched for a product. • Product comparison data has already been collected by the system.
Postconditions	If complete data is available, product comparison is displayed. If data is not available, no comparison is shown.
Main Success Scenario	1. User selects a product from the product results. 2. System receives the selected product information and comparison data. 3. System verifies that comparison data is complete for all sources. 4. System displays the product comparison to the user.
Alternative Scenarios	• 4a. Comparison Data Not Available: If data is incomplete, the system does not display the comparison .
Frequency	Medium

Table 2.3: Use Case Description: View Product Comparison

Use Case ID	UC-04
Use Case Name	Filter and Sort Products
Primary Actor	User
Description	The user to refine the product list by applying filter criteria (e.g., category, price) and sorting options.
Preconditions	The user has performed a product search.
Postconditions	Filtered or sorted products are displayed.
Main Success Scenario	1. User views the product search results. 2. User selects one or more filter options. 3. User selects a sorting option. 4. System applies the selected filter criteria. 5. System sorts the filtered product list. 6. System displays the updated filtered and sorted products.
Alternative Scenarios	• 3a. User Selects Only Sorting: System sorts complete list without filters. • 4a. No Products Match Filter: System displays "No matching products"
Frequency	Low; Frequently used after performing a product search.
Special Requirements	• Filters must be easy to apply and remove.

Table 2.4: Use Case Description: Filter and Sort Products

2.9 System Sequence Diagram

Search Product:

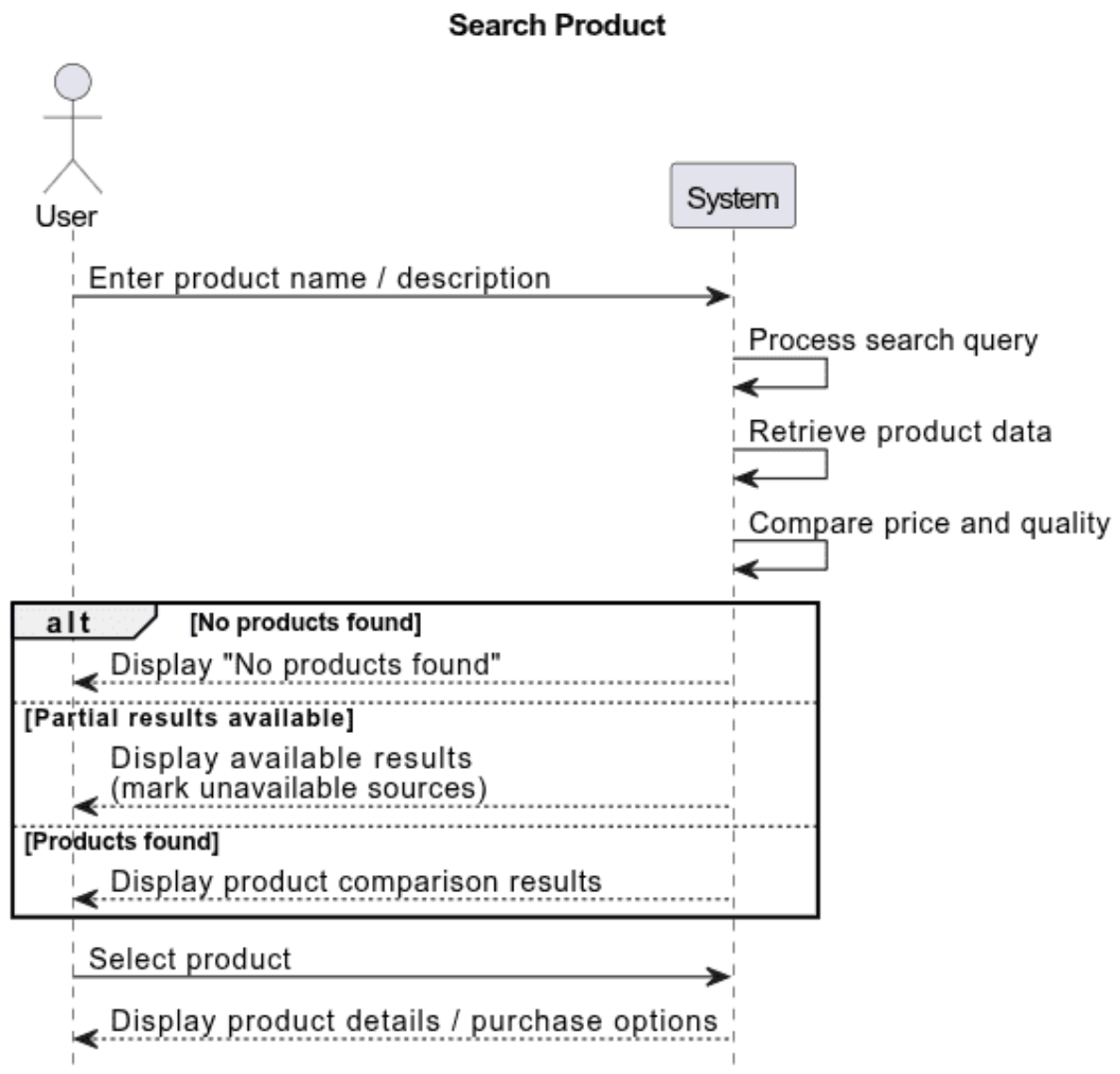


Figure 2.2: System Sequence Diagram: Search Product

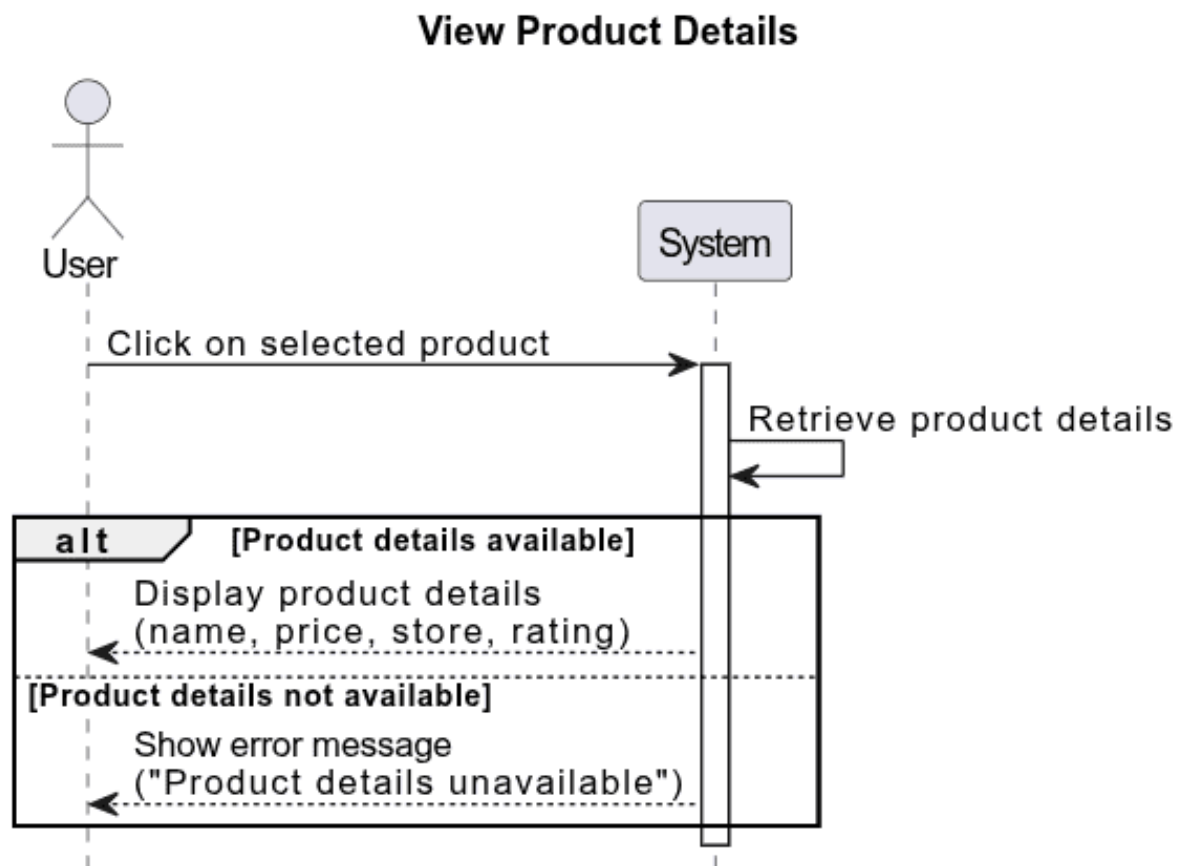
View Product Details:

Figure 2.3: System Sequence Diagram: View Product Details

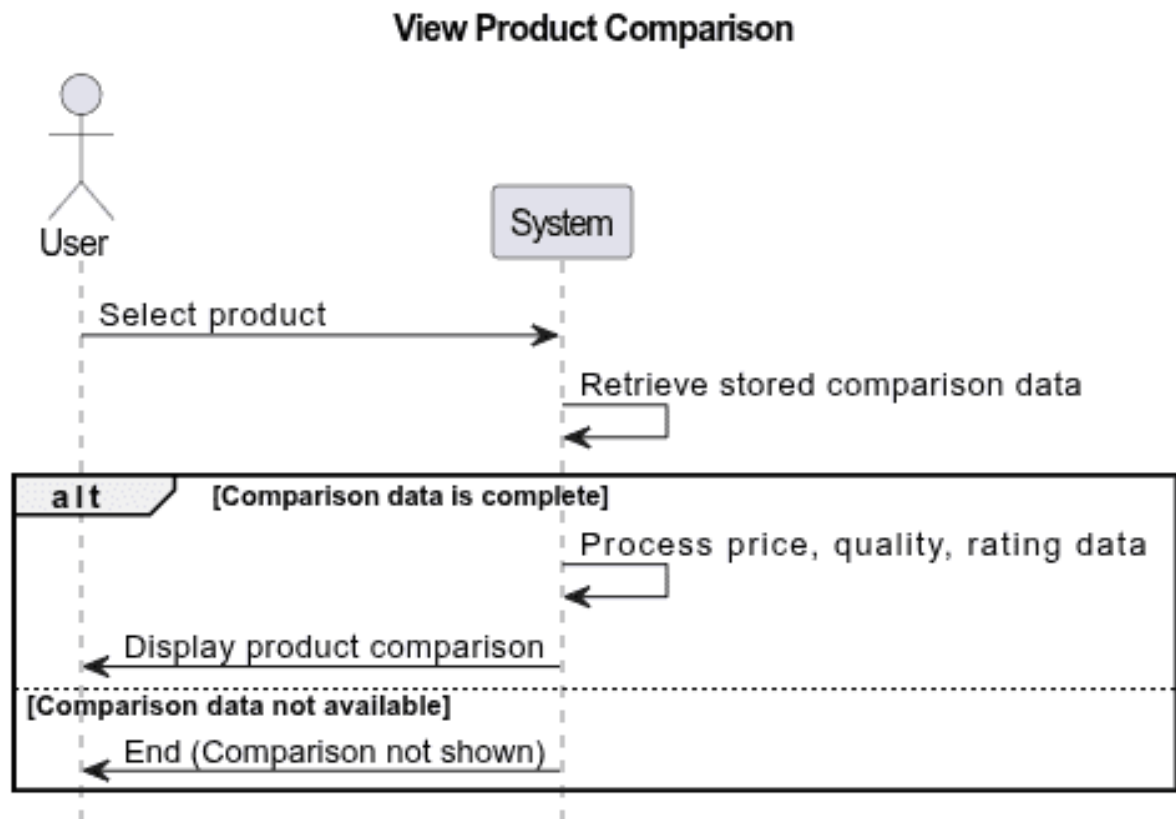
View Product Comparison:

Figure 2.4: System Sequence Diagram: View Product Comparison

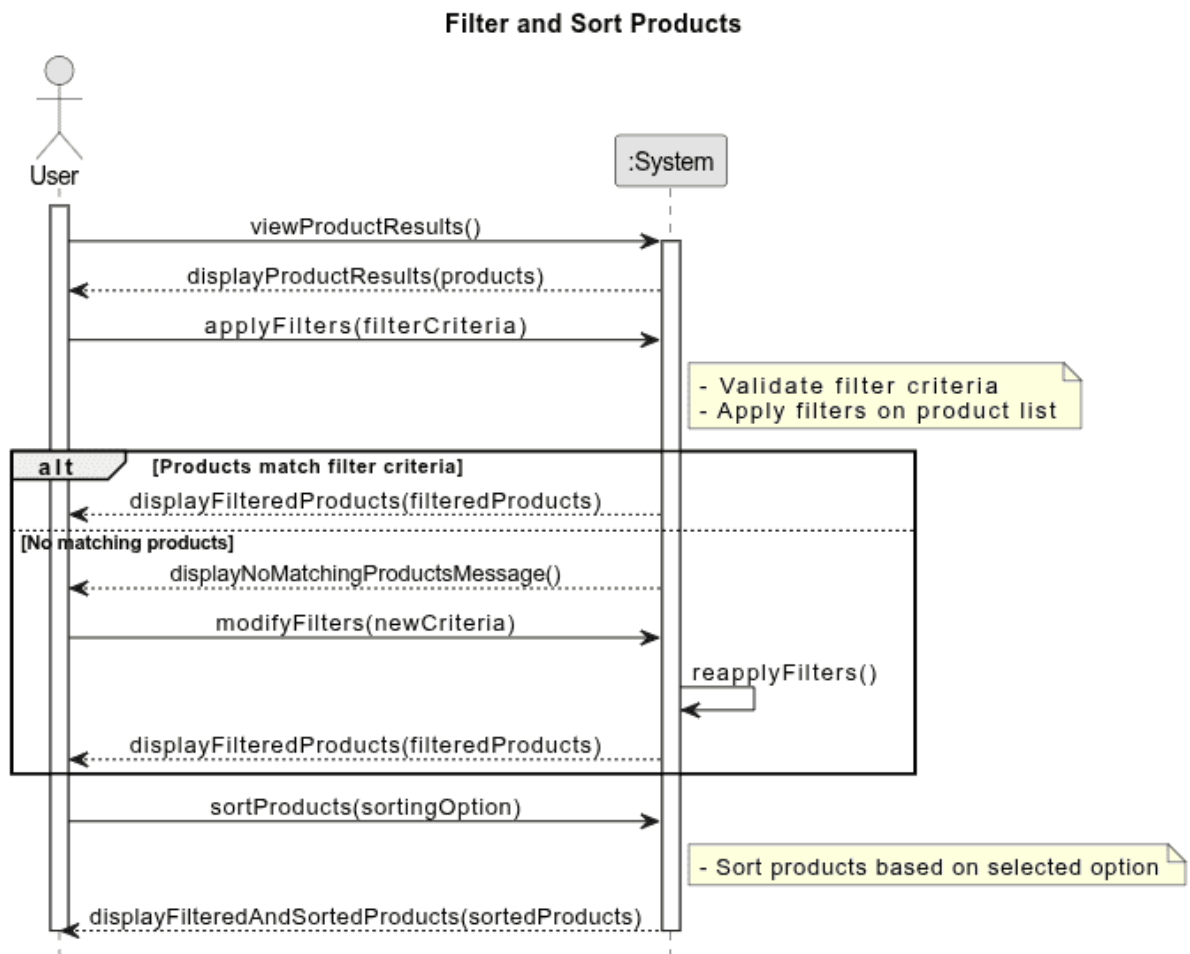
Filter And Sort:

Figure 2.5: System Sequence Diagram: Filter And Sort

2.10 Activity Diagrams

1. Search Product Workflow
2. View Product Details
3. View Product Comparison
4. Filter and Sort Products Logic

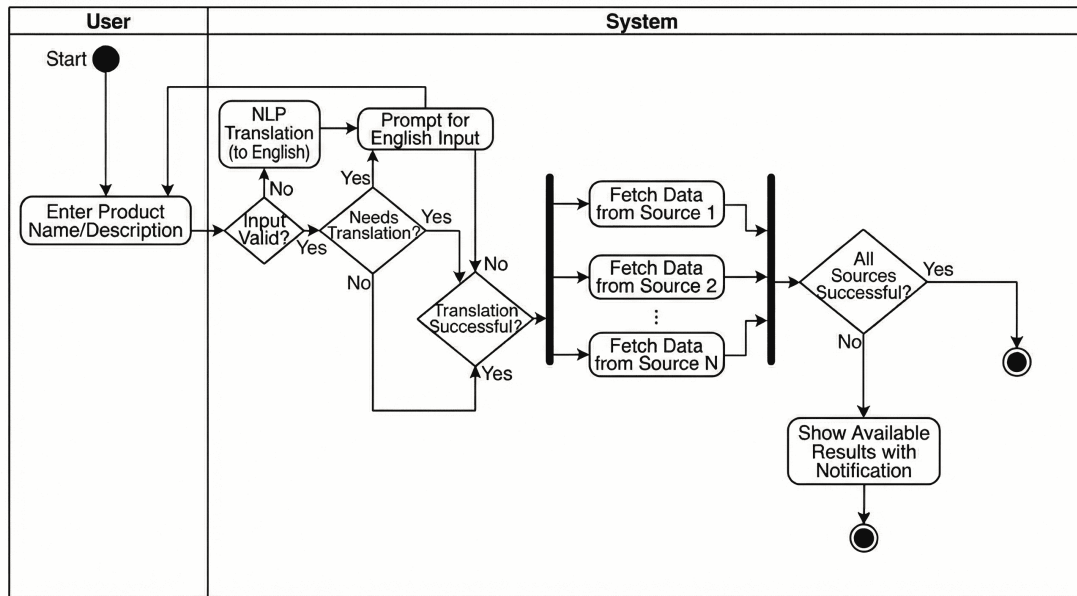
Search Product:

Figure 2.6: Activity Diagram: Search Product

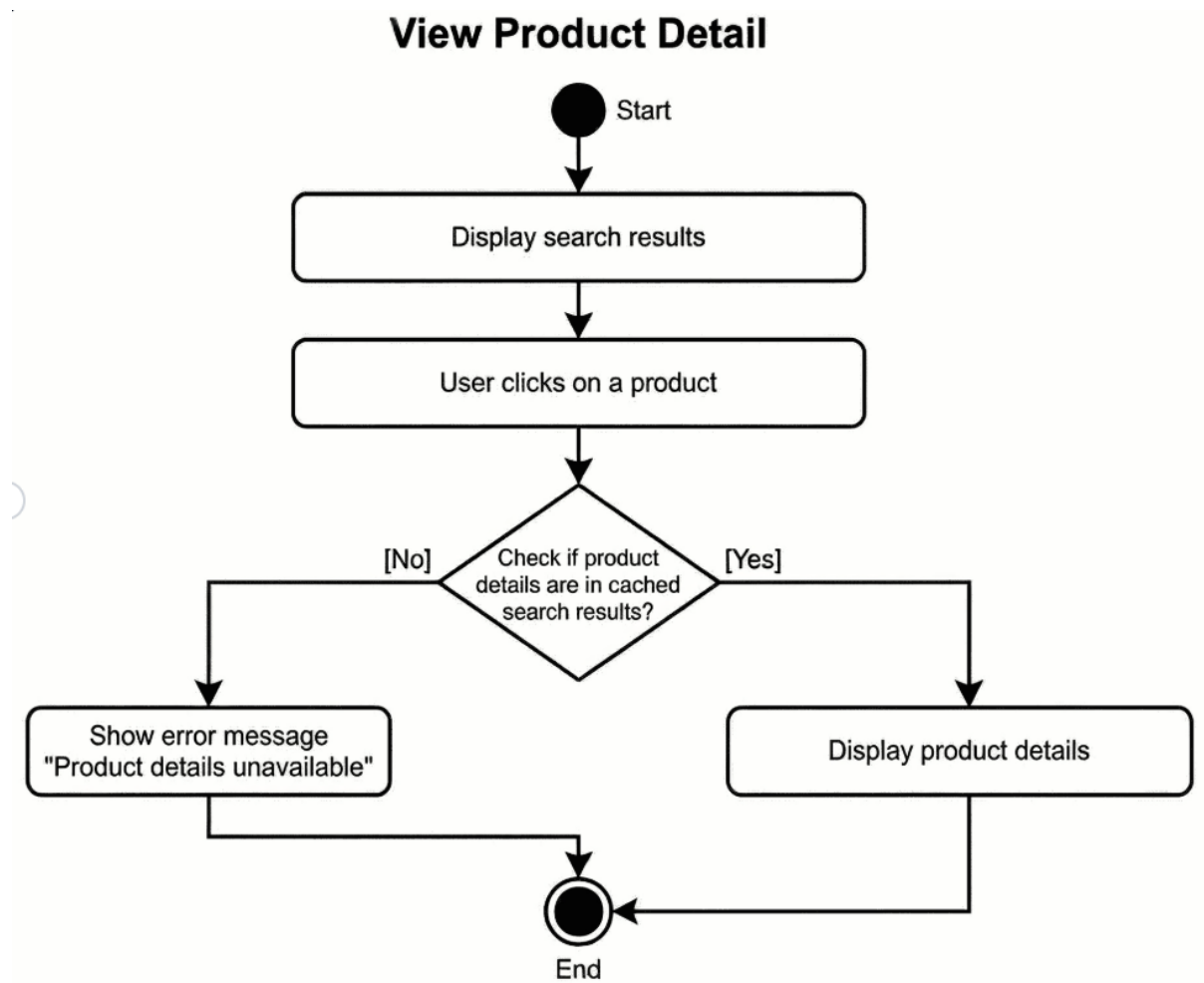
View Product Details:

Figure 2.7: Activity Diagram: View Product Details

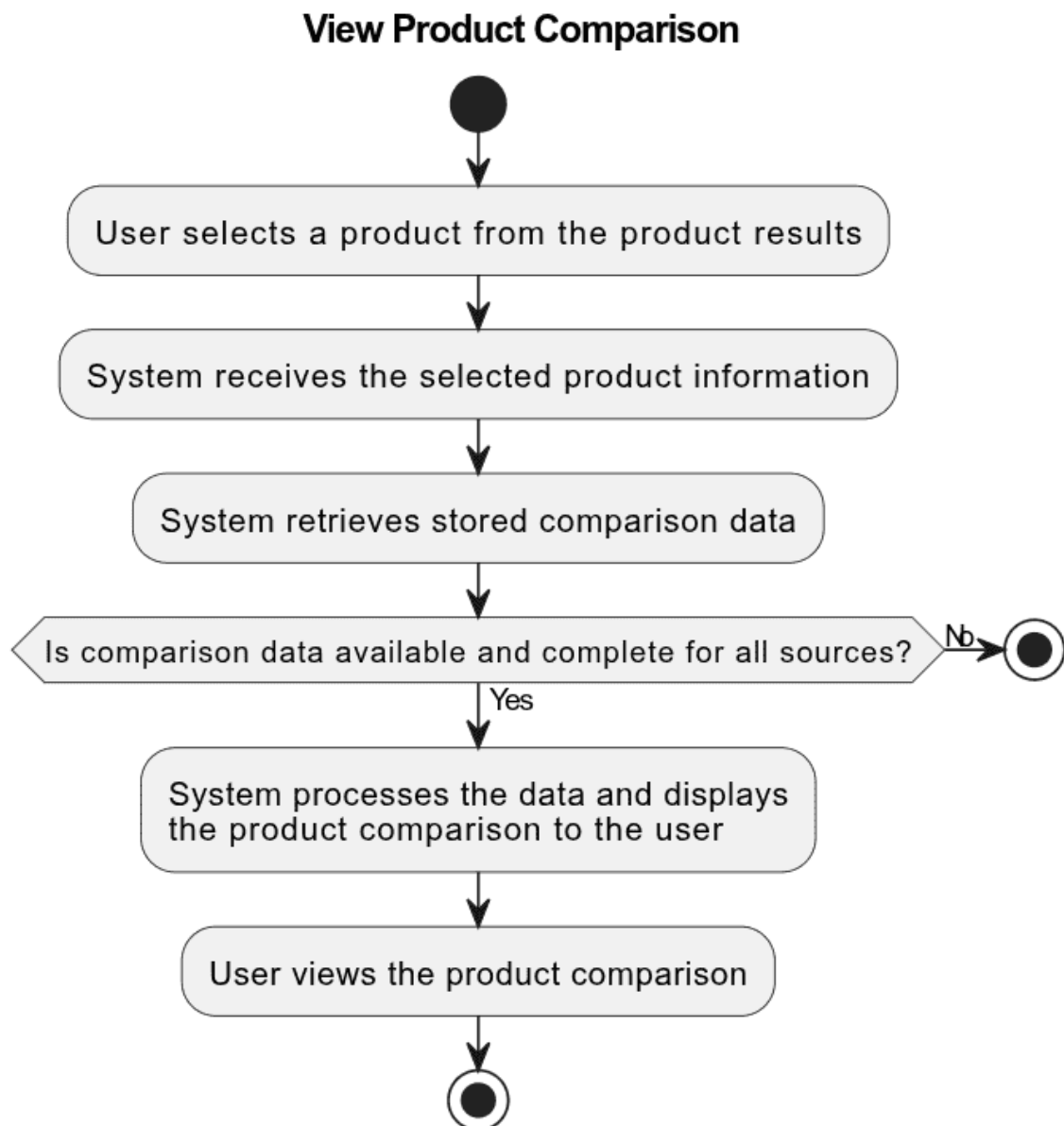
View Product Comparison:

Figure 2.8: Activity Diagram: View Product Comparison

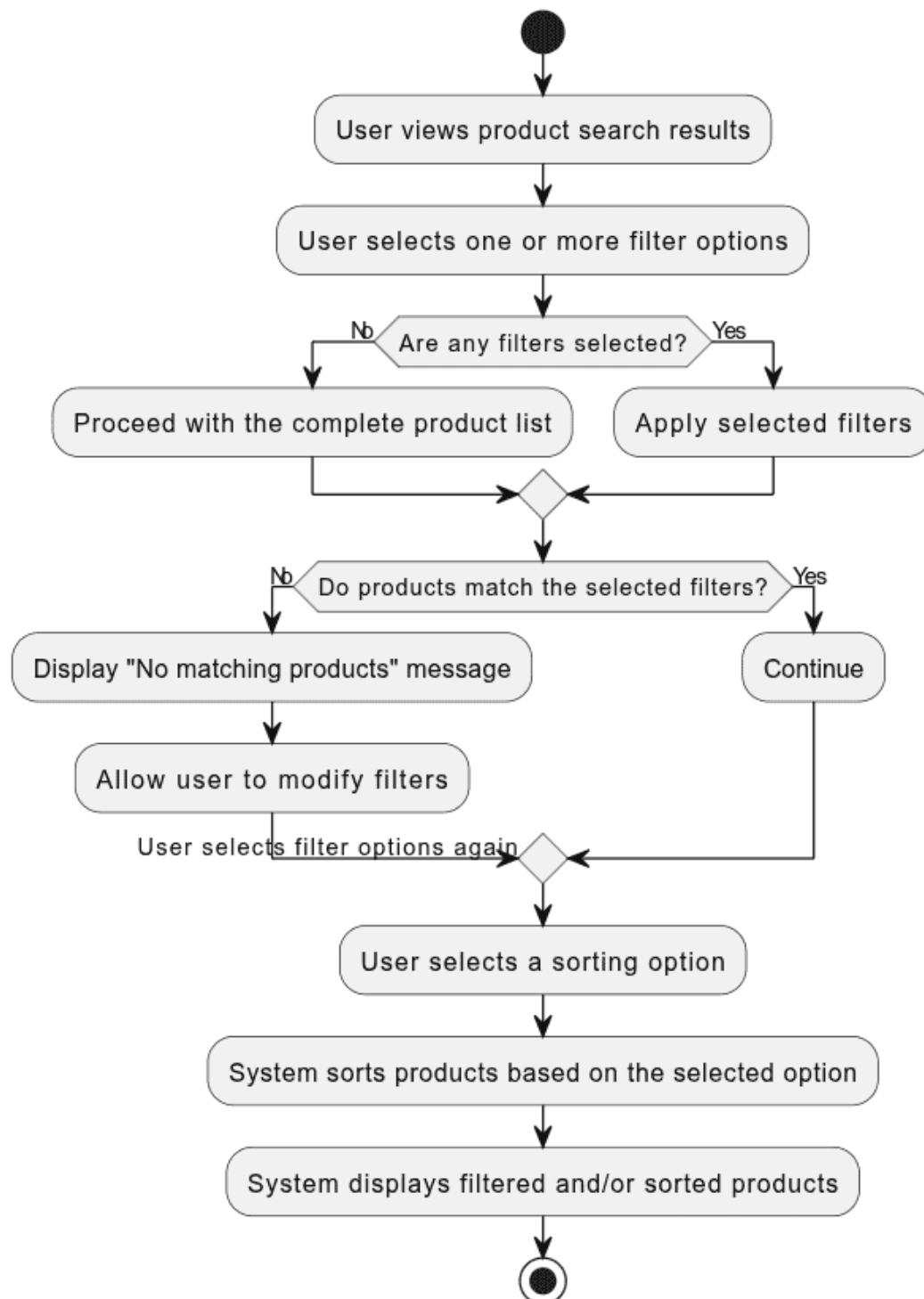
Product Filter And Sort:

Figure 2.9: Activity Diagram: Filter And Sort

2.11 Domain Model

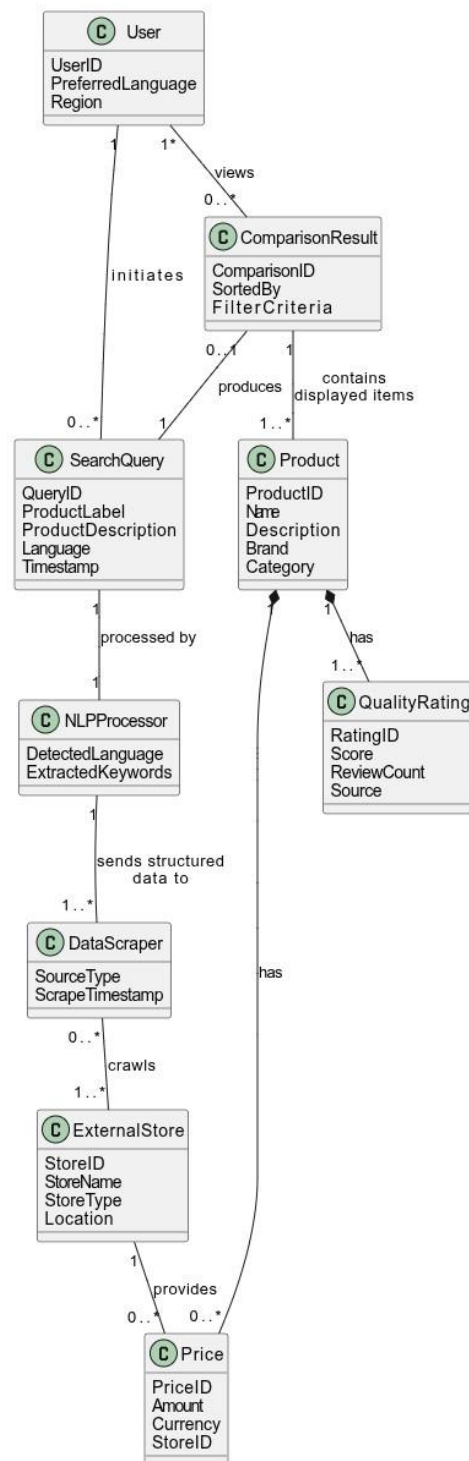


Figure 2.10: Domain Diagram

CHAPTER 3 Software Design Description (SDD)

3.1 Introduction

This chapter provides an in-depth insight of the design and architecture of the Comparison System between the Product Price and Quality. The functional is then transformed to the design stage non-functional requirements located in Chapter 2 in an actual system blueprint.

3.1.1 Design Overview

It is a **Product Price and Quality Comparison System (PQC)** by web, which is a modular one developed in a modern full-stack style

- The frontend to be a responsive and interactive user interface based on **Next.js (React)** for the frontend to ensure a responsive and interactive user interface.
- The backend that is being driven by the **Node.js** is handling the natural language **Web Scrapping** modules
- The **MongoDB** is implemented as the central database onto which the user logs, product information (cached) are deposited and comparison history.

3.1.2 Design Patterns Implementation

- **Model-View-Controller (MVC):** To separate the products data models, the interface scraping and filtering control logic, and the views.
- **Strategy Pattern:** The Web Scrapping Engine uses the strategy to handle different scraping for e-commerce plans of various sites.

3.1.3 Requirements Traceability Matrix (RTM)

Requirement ID	Use Case / Requirement	Mapped Test Cases	Sequence Diagram
UC-01	Search Product (NLP & Scraping)	TC-FUNC-01, TC-FUNC-02, TC-NLP-01	SSD-01
UC-02	View Product Details	TC-FUNC-03	SSD-02
UC-03	View Product Comparison	TC-COMP-01	SSD-03
TC-01	Performance Testing	TC-PERF-01	SSD-01

Table 3.1: Requirements Traceability Matrix

3.2 System Architecture Design

This section describes the architectural structure of the system, including its major components and their relationships.

3.2.1 Architectural Overview

The system conforms to the client-server form of architecture with the client playing the role of user interaction and server side application logic and data processing. The architecture will support simultaneous users, efficient data handling and secure interaction between the components.

- Scalability for increasing user requests
- Maintenance Modular design.

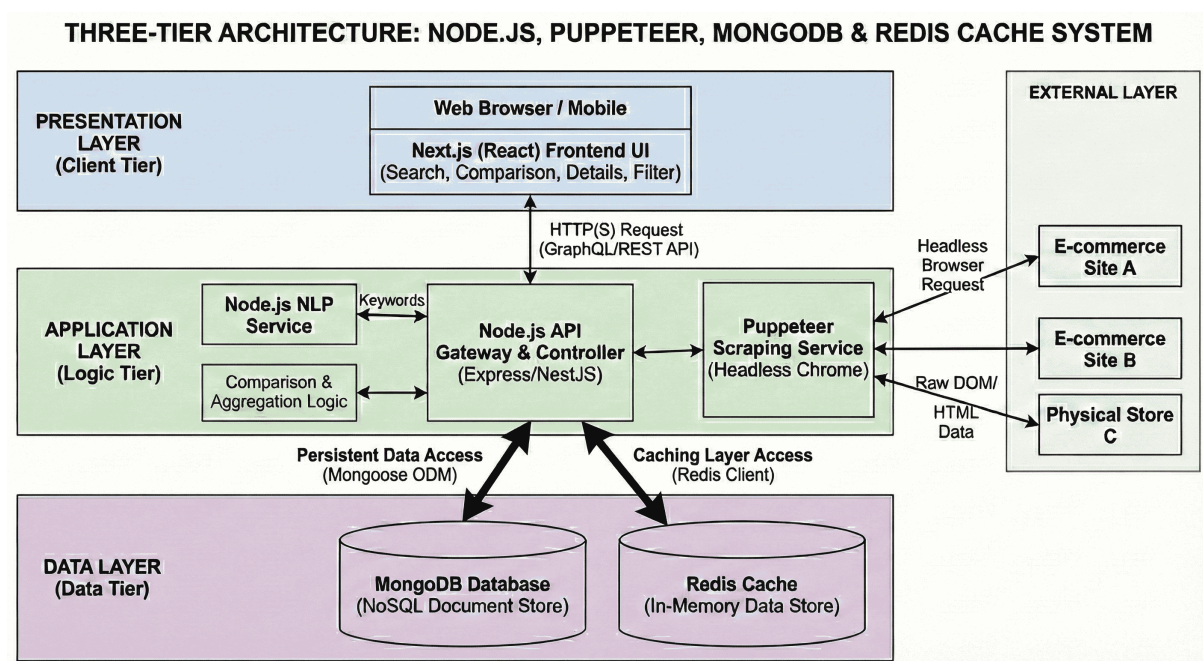


Figure 3.1: Three- Tier Architecture

3.2.2 Architectural Layers

1. Presentation Layer

- In charge of user interaction.
- Provides responsive and user-friendly interface.

2. Application Layer

- Both of them contain the business logic.
- Processes user requests
- Co-ordinations between the presentation and data layer.

3. Data Layer

- Takes care of storage and retrieval of data.
- Provides data-integrity and consistency.
- Processes database activities.

3.2.3 System Components Interaction

1. The process starts at the **Presentation Layer**, Presentation Layer, whereby the user sends a request.
2. The request is forwarded to the **Application Layer**, which contains the business logic and coordinates the necessary operations. This request gets sent to the **Application Layer** that has the business logic and orchestrates the requisite operations.
3. In case of need of access to data, the Application Layer interacts with a **Data Layer** to either store or retrieve data.
4. The results of the process are processed and sent back through the Application Layer to the Presentation Layer.
5. Lastly, the Presentation Layer interpretes the response and presents the response to the user.

The following benefits can be offered by this interaction model:

- **HTTP/REST APIs:** Dialogue between Presentation and Application layers.
- **ORM Queries:** Application-data Layer Communication.
- **Internal Service Calls:** Interapplication communication.

3.3 System Sequence Diagram

Search Product:

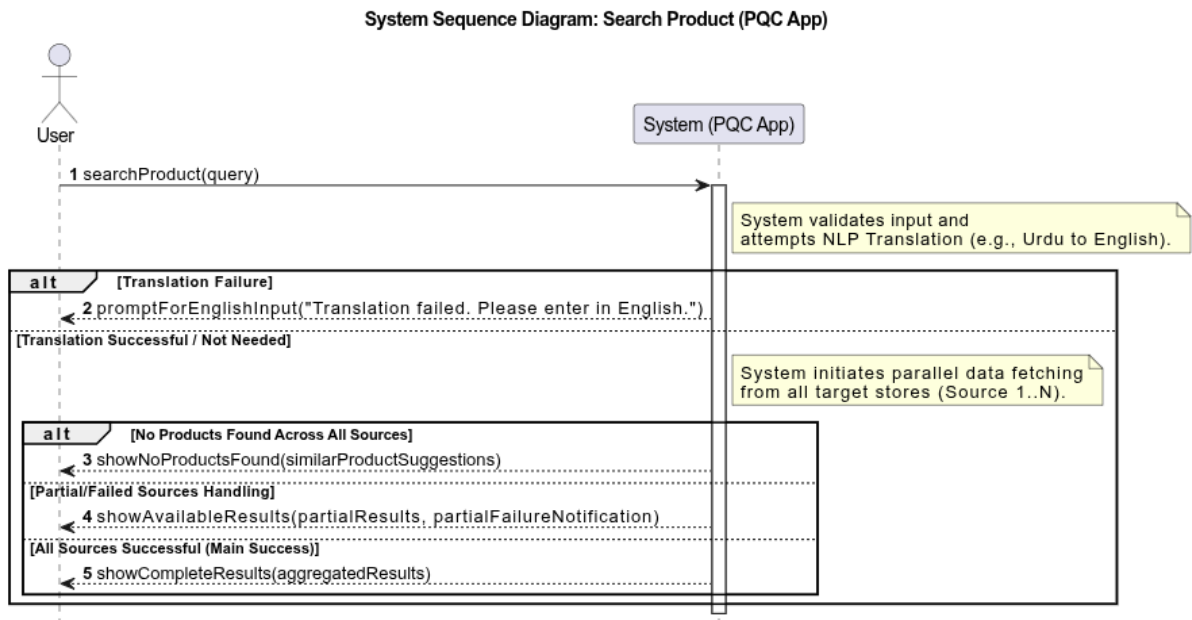


Figure 3.2: Sequence Diagram: Search Product

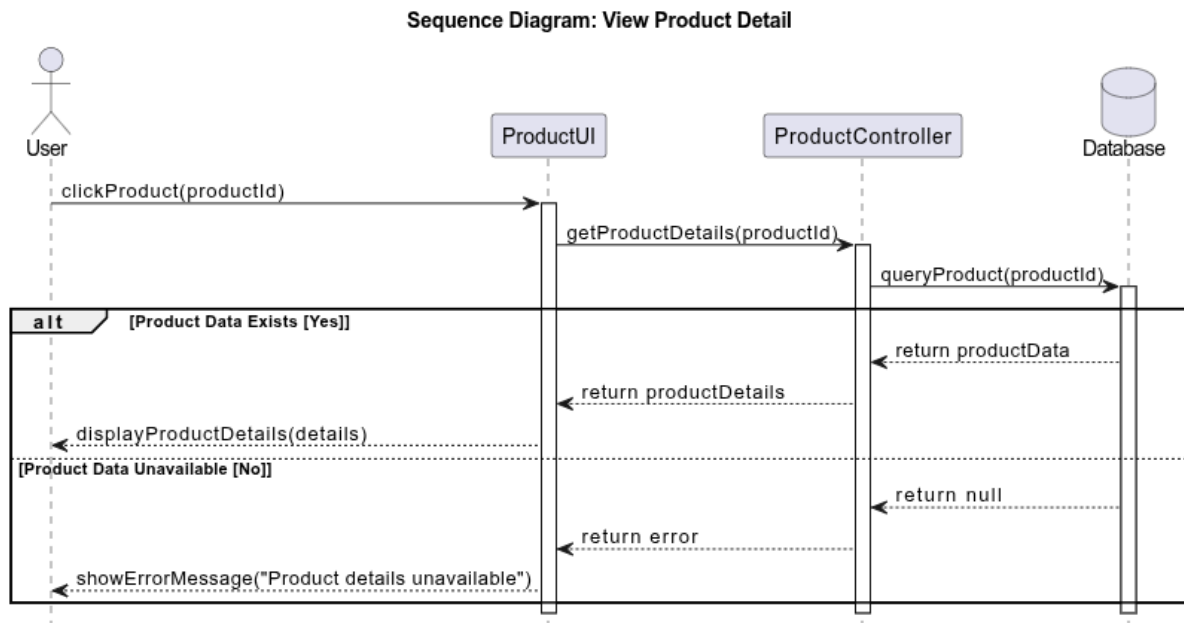
View Product Details:

Figure 3.3: Sequence Diagram: View Product Details

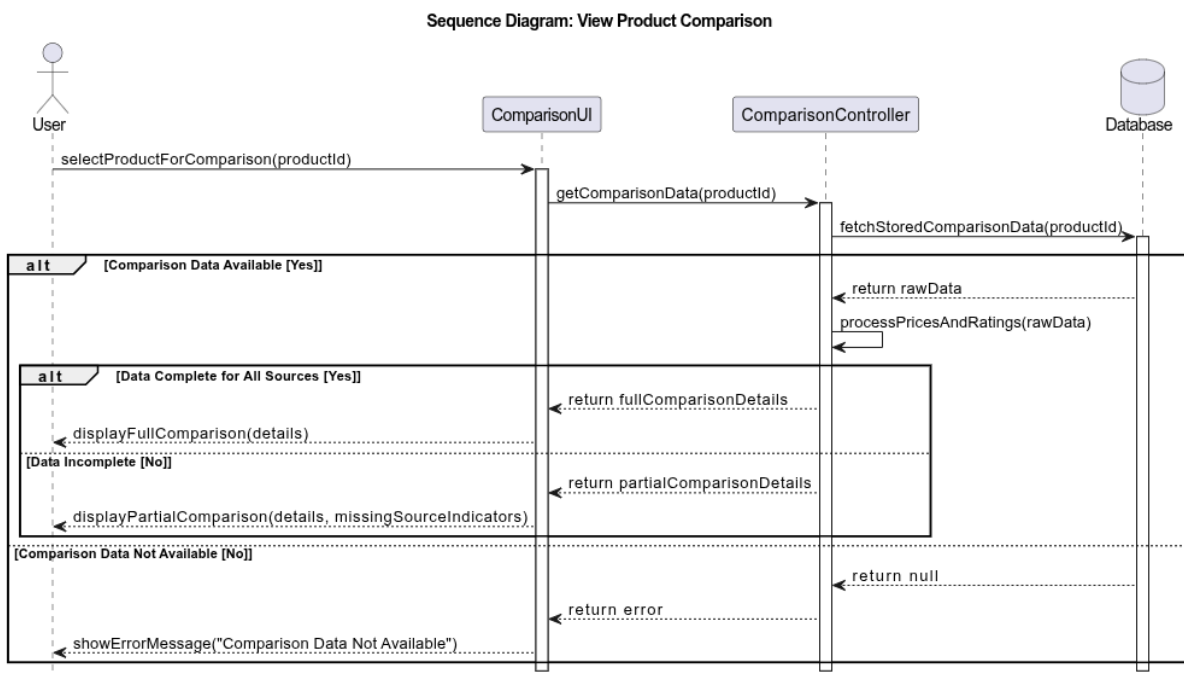
View Product Comparison:

Figure 3.4: Sequence Diagram: View Product Comparison

Product Filter And Sort:

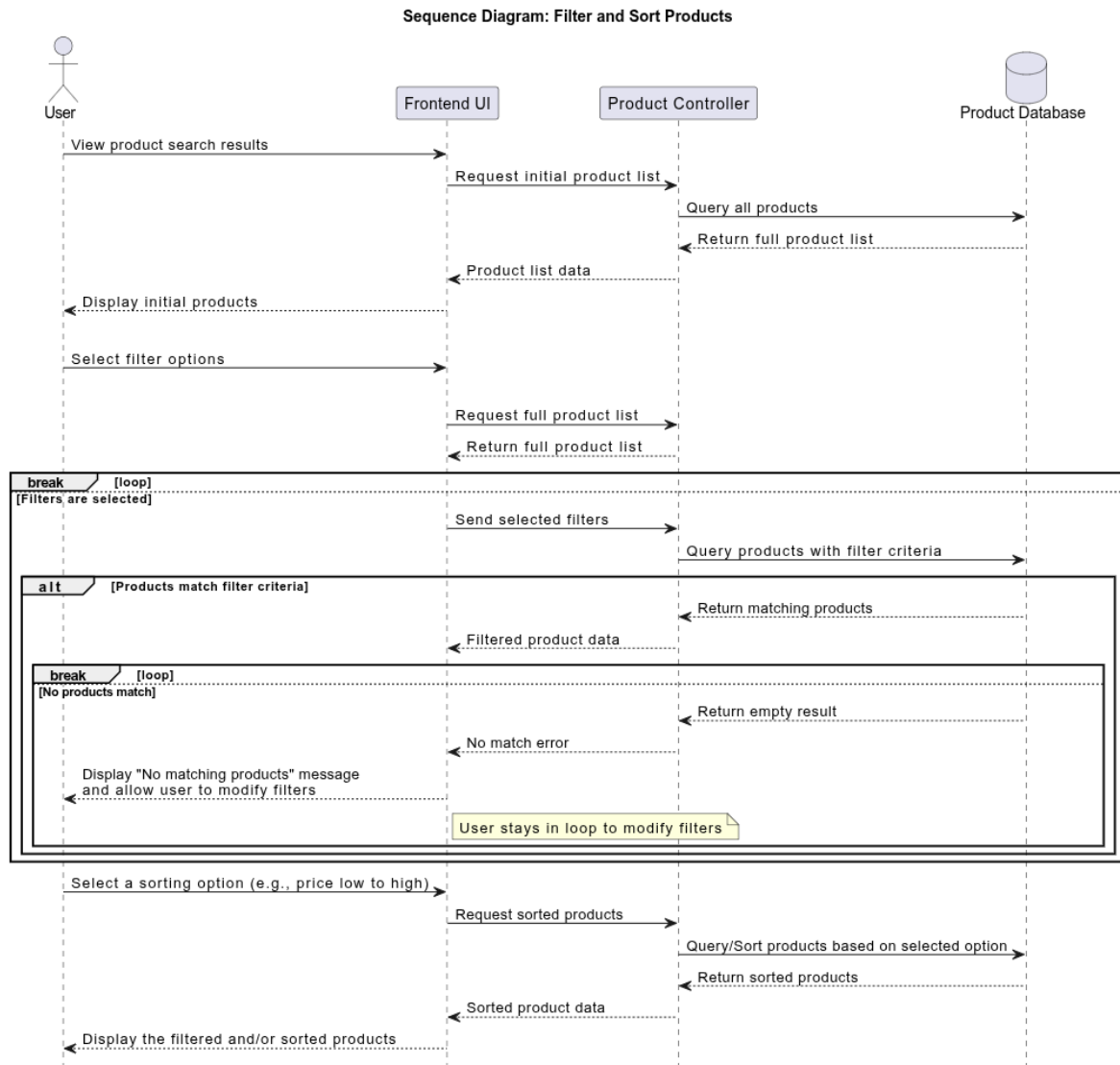


Figure 3.5: Sequence Diagram: Filter And Sort

3.4 Class Diagram

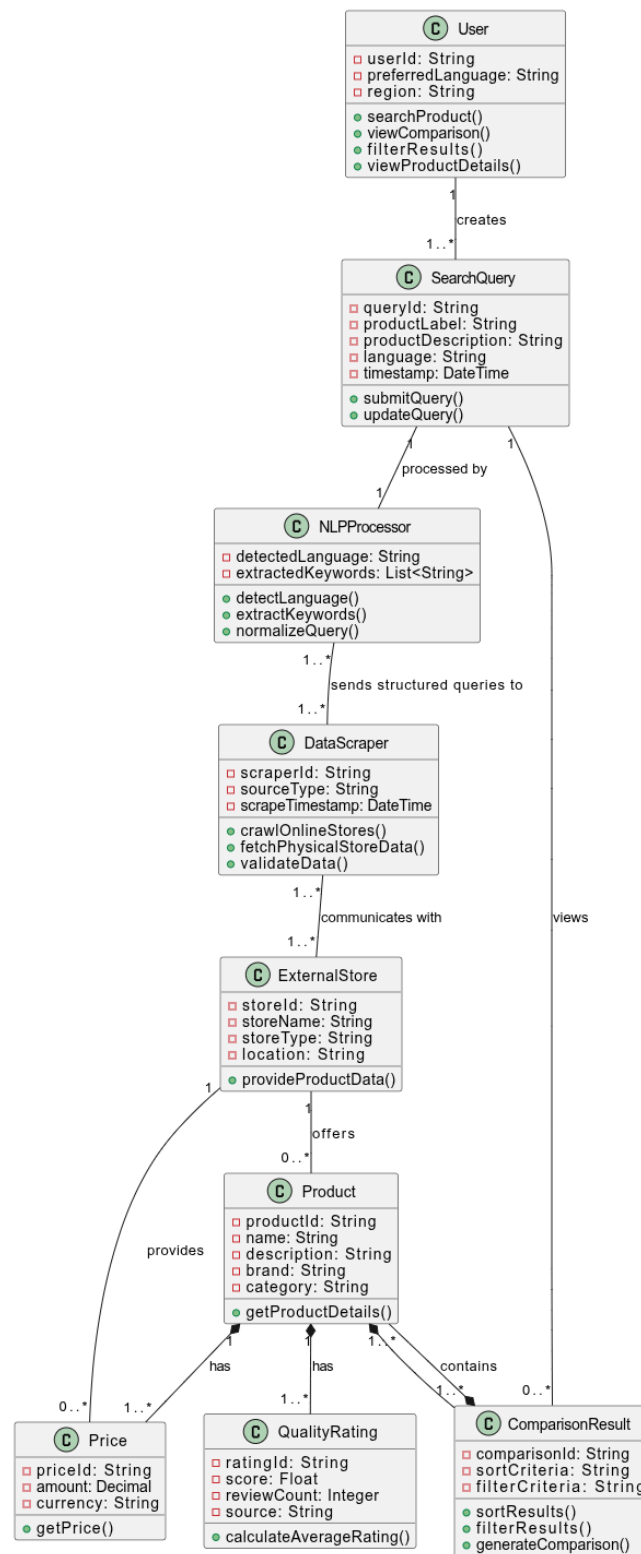


Figure 3.6: Class Diagram

CHAPTER 4 Software Implementation (SI)

4.1 System Definition

A focus first takes an interest in the design of the system, in its requisite elements unfolding afterward. There are requirements that follow, which are defined by the desired functions as opposed to abstract concepts. Next is functionality which is incorporated in each layer rather than being added afterwards. Organization leads every decision, to prepare sanity a consistent result. External components get connected with the system as the user does. These kinds of systems as web scrapers and natural language processors are indirect in their interactions, influencing the manner in which information is circulated by different channels, but observed to be connected.

4.2 Development Tools

- **Frontend:** Next.js for responsive UI and fast rendering
- **Backend:** Node.js with Express.js for API development
- **Database:** MongoDB to store the information about products, the results that are cached, comparison information.
- **Web Scraping Engine:** Puppeteer to scrap product information on dynamically loaded websites.
- **NLP Module:** This is generated to use in query normalization, translation (Urdu ↔ English) and keyword extraction.
- **Version Control:** Git
- **Tailwind CSS:** CSS Framework Reasonable effort is required to make the site look good across all devices (phones, tablets, computers).
- **Bootstrap:** Other design elements.
- **JavaScript:** Makes the website Logic and UI
- **Testing:** Functional testing with failure processing Testing: Manual testing of functional accuracy and failure handling

4.3 Implementation Details

The system is implemented using a modular architecture where each module corresponds directly to system use cases.

4.3.1 User Interaction Module

The interactions between users of this system are mostly done with this module. The users type product questions, scroll through lists of results, choose products they desire, filter and browse products being compared. The search terms entered by the users are verified to avoid blank or wrong searches allowed. The module itself is dedicated to features such as the search of product, the viewing of what is inside of a product, and *filtering or sorting* the products that people would like to see.

4.3.2 Query Processing (NLP) Module

Inside this module, the description of user or name of a product is processed. It handles making input normal, identifying language and also converting Urdu questions if needed into English. Important words are taken out for increasing matching quality. When the NLP part cannot deal with the question, system asks users about rephrasing the entered input, and this follows the case covered with alternate situations for *Search a Product*.

4.3.3 Web Scraping and Data Retrieval Module

This module gathers product data from lot of online and physical store places with methods of automated web scraping. The data pulled contains prices, reviews, if the product is in stock and information about specific shop. The system is doing its best when facing some failures; it can show results from sources that worked and points out which source could not be found.

4.3.4 Product Matching and Comparison

The engine for matching and comparing products checks the data fetched, focusing on how similar items are, the cost, and with quality points. It is controlling so that comparing products are shown only if complete details are available from all sources as described earlier in product comparing cases. Products that were matched are ordered and made ready for display with the comparison that is formatted in structured way.

4.4 Data Flow

The query is submitted to NLP processing and submitted to scraping engine. The system puts reads data in database in a temporary basis, and then transfers it to matching and comparison steps. Real time filtered, sorted and compared results are displayed to user and cache data is also to examine product information.

4.5 Summary

This chapter has described way the PQC system is made and how it is directly following its functional cases. Using different modules leads to an efficient product searching, correct comparison and a smooth way for users to interact. Joining NLP with web scraping and live filtering, PQC system completely does its work for trustworthy price and quality comparison.

CHAPTER 5 Software Test Documenta- tion (STD)

5.1 Introduction

Software testing plays a key role when developing Price and Quality Comparison (PQC) system. This chapter tries to show what actions were carried out during the test phase regarding both functioning and also not functioning aspects. Software testing was done to confirm that the operations of PQC system are correct, system is reliable, easy to use and it has a high strong.

5.1.1 Test Approach

The PQC system had a functional and scenario-based testing approach. Manual testing was mainly adopted because of the dynamic web scraping and real time information retrieval. The testing process involved unit testing, integration testing, system testing and user acceptance testing to test the stability of the system in general.

5.2 Test Plan

The test plan defines the strategy, objectives, scope, tools, and schedule used during testing of the PQC system.

5.2.1 Testing Objectives

- To ensure that every system use case is working properly.
- In order to have the right search and comparison results of the products.
- To confirm the multilingual query processing.
- To confirm handling invalid inputs
- To guarantee quick response time and easy user experience.

5.2.2 Testing Scope

- Searching of products.
- NLP-based query processing
- Data retrieval and web scraping.
- Matching and comparison of products.
- Filter and sort operations
- Responsiveness of user interface.

5.2.3 Testing Tools and Environment

Tests were conducted on the following environment and tools:

- Operating System: windows/ Linux.
- Web Browser: Google Chrome
- Frontend Framework: Next.js
- Backend Framework: Node.js
- Database: MongoDB
- Web Scraping Tool: Puppeteer

5.2.4 Testing Schedule

The testing activities were carried out iteratively during the development lifecycle.

Phase	Testing Activity
Module Completion	Unit testing of individual modules
Feature Integration	Integration testing between modules
System Completion	System and user acceptance testing

Table 5.1: Testing Schedule

5.3 Test Cases

The following test cases were designed and executed based on the system use cases.

5.3.1 Test Case: Valid Product Search

Test Case ID	TC-FUNC-01
Requirement ID	UC-01
Objective	To verify that the system correctly displays relevant products when a valid search query is entered.
Precondition	User is on the product search interface.
Test Steps	1. Enter valid product name. 2. Click search button.

Expected Result	Relevant products along with price and quality information are displayed.
Actual Result	Product list displayed correctly with comparison data.
Status	Pass

Table 5.2: Functional Test Case: Valid Product Search

5.3.2 Performance Testing

Test Case ID	TC-PERF-01
Description	Measure response time when fetching product data from a single store source
Tester	Student
Expected Result	System should retrieve and display product data from one store within 2 seconds
Actual Result	Product data from a single store was fetched and displayed in approximately 1.6 seconds
Status	Pass

Table 5.3: Test Case: Single Store Response Time

5.3.3 Test Case: View Product Details

Test Case ID	TC-FUNC-03
Requirement ID	UC-02
Objective	Detailed product information display functionality.
Expected Result	Complete product details are displayed.
Actual Result	Product details displayed successfully.
Status	Pass

Table 5.4: Functional Test Case: View Product Details

5.3.4 Test Case: Product Comparison

Test Case ID	TC-COMP-01
Requirement ID	UC-03

Objective	To verify product comparison functionality across multiple sources.
Expected Result	Comparison displayed only if complete data is available.
Actual Result	Comparison displayed correctly.
Status	Fail

Table 5.5: Functional Test Case: Product Comparison

5.3.5 Test Execution Summary

Total Test Cases	Passed	Failed
4	3	1

Table 5.6: Overall Test Execution Summary

5.4 Conclusion

In the testing process, it was confirmed that the PQC system along with its functional requirements met and forms under normal and exceptional conditions are reliable. The major use cases were all successfully tested, showing the correctness, stability and usability of the system.

CHAPTER 6 Conclusion and Future Work

6.1 Conclusion

This project had been able to come up with a web based Price and Quality Comparison (PQC) system that allows the search, comparison, and analysis of products of various online and physical stores. The system combines query processing with NLP, automatic web scraping and smarter comparison algorithms to provide precise and easy to use results. The system is scalable because of the use of architecture, multilingual support, and real time filtering features and efficient

6.2 Future Work

The PQC system meets its current goals, several improvements can be done in the future:

- Integration of additional online and physical store sources
- Scrape the data from Facebook and TikTok for small e-commerce stores
- User accounts with personalized preferences and history
- Development of a mobile application version
- Performance optimization for large-scale usage

These improvements would further enhance the functionality, accuracy, and scalability of the system.